GPU-Accelerated 3D Point Cloud Histogram Equalization

COL380 Assignment 2 – CUDA Implementation and Analysis

Shahid Khan
ee1221163

Indian Institute of Technology Delhi
Department of Computer Science and Engineering

March 27, 2026

Contents

1 Introduction **3**

2 Algorithm Overview **3**
2.1 Histogram Equalization Helper . . . 3
2.2 Exact KNN . . . 3
2.3 Approximate KNN via Uniform Grid . . . 4
2.4 Approximate KNN via Spatial Hashing . . . 4
2.5 K-Means Clustering . . . 4

3 GPU Optimisations **4**
3.1 Shared-Memory Tiling (Exact KNN) . . . 4
3.2 Spatial Hashing (Approximate KNN) . . . 5
3.3 Implementation Selection and Consolidated Binary . . . 5
3.4 Compiler-Level Optimisations . . . 5
3.5 Heap vs. Unsorted Array for Exact KNN . . . 5

4 Experimental Setup **6**

5 Results and Analysis **6**
5.1 Scalability with N (Group A) . . . 6
5.2 Tiled vs. Non-Tiled Exact KNN . . . 7
5.3 Scalability with k (Group B) . . . 8
5.4 K-Means Iterations T (Group C) . . . 8
5.5 Data Distribution Effects (Group D) . . . 8
5.6 Spatial Hash vs. Grid Approximate KNN . . . 9
5.7 Coordinate Range Effects (Group G) . . . 9
5.8 Large-Scale Benchmarks (Group F) . . . 10
5.9 Correctness Verification (Group E) . . . 10

6 Discussion **11**
6.1 Summary of Optimisation Gains . . . 11
6.2 Register Pressure and Tiling Limits . . . 11
6.3 Approximate KNN Accuracy . . . 11
6.4 Spatial Hashing vs. Grid: When to Use Which . . . 12

7 Conclusion **12**

1 Introduction

Histogram equalization is a classical image-processing technique that improves contrast by redistributing intensity values so that the empirical CDF of the output is approximately uniform. This assignment extends that idea to a 3-D point cloud: each point p_i carries an intensity $I_i \in [0, 255]$, and its equalized intensity is computed from the local neighbourhood histogram using

$$I_i' = \left\lfloor \frac{\big(\mathrm{CDF}(I_i) - \mathrm{CDF}_{\min}\big) \cdot (L-1)}{m - \mathrm{CDF}_{\min}} \right\rfloor, \qquad L = 256, \tag{1}$$

where the CDF counts points (including p_i itself) in the local window with intensity $\leq I_i$, $\mathrm{CDF}_{\min}$ is the first non-zero local CDF value, and m is the window size. If $m = \mathrm{CDF}_{\min}$, we keep $I_i' = I_i$.

Three neighbourhood definitions are implemented and benchmarked:

1. **Exact KNN** — brute-force all-pairs $O(N^2)$ distance computation to find the k nearest neighbours.
2. **Approximate KNN** — grid-based expanding shell search that terminates as soon as k neighbours are found; sub-linear in practice.
3. **K-Means Clustering** — assigns points to one of k clusters, uses cluster members as the neighbourhood.

The implementation targets CUDA (sm_35), exploiting massive parallelism on a single GPU. Three algorithmic optimisations were prototyped and evaluated:

- **Shared-memory tiling** for the exact KNN kernel, reducing global memory bandwidth by a factor of `BLOCK_SIZE` = 128.
- **Spatial hashing** for the approximate KNN, replacing a flat 3-D grid array (capped at 10^7 cells) with a compact chained hash table of size $2N$.
- **Unsorted array with dynamic CDF** for exact KNN, replacing the max-heap with an $O(k)$ linear-scan structure and the 256-bin histogram with an inline CDF computation.

Five binary variants were compiled from a single source file via preprocessor macros for a controlled comparison; benchmarking identified the best-performing combination, which was then consolidated into a single binary `a2` with a three-mode CLI (`knn`, `approx_knn`, `kmeans`).

2 Algorithm Overview

2.1 Histogram Equalization Helper

Each GPU thread calls `equalize_point(hist, I, m)`, which sweeps through the 256-bin histogram accumulated from the local window to compute the CDF value at I, tracks the first non-zero local CDF value, and applies Equation (1). The histogram is stored entirely in thread-private registers.

2.2 Exact KNN

Each thread i scans *all* N points, maintaining a max-heap of the k smallest squared Euclidean distances. The heap lives in registers (`long long heap_dist[MAX_K]`, `int heap_idx[MAX_K]`) plus a 256-bin intensity histogram – together ≈ 4.5 kB per thread. After the scan, the histogram is used to equalize p_i. Complexity: $O(N^2)$.

2.3 Approximate KNN via Uniform Grid

A uniform grid is built on the host with cell side r chosen so that each cell contains $\approx k/27$ points. Point indices are sorted into the grid using a counting sort. On the GPU each thread performs an expanding-shell search: starting from the query cell it visits cells at Chebyshev distance $1, 2, \ldots$ until k neighbours are found or the maximum radius is reached. An early-exit warp vote (`__all_sync`) stops the expansion once the entire warp has found k neighbours. Complexity: sub-linear on average.

2.4 Approximate KNN via Spatial Hashing

The same expanding-shell logic is used, but the grid array (of potentially $> 10^7$ entries) is replaced by a chained hash table of size $2N$ (next power-of-2 $\geq 2N$). The hash function

$$h(c_x, c_y, c_z) = \big((c_x \cdot 73856093) \oplus (c_y \cdot 19349663) \oplus (c_z \cdot 83492791)\big) \bmod \texttt{ht_size} \tag{2}$$

maps integer cell coordinates to a bucket. Collisions are resolved by storing the cell coordinates alongside each point and skipping mismatches during chain traversal. With load factor 0.5 the expected chain length is ≤ 1, so most bucket lookups cost a single memory access, and the entire hash table (≈ 2.4 MB for $N = 100\,000$) fits in the GPU's L2 cache. Unlike the grid version, no bounding-box cap is needed: out-of-range cells simply have no entries (`head = -1`).

2.5 K-Means Clustering

k cluster centres are initialised from the first k distinct points. Each of T iterations runs three CUDA kernels:

1. **Assign** – each point picks the nearest centre (shared memory caches the k centres). An `atomicExch` flag detects convergence on-device, avoiding a per-iteration PCIe download.
2. **Accumulate** – `atomicAdd` into per-cluster coordinate sums (stored as `unsigned long long` to exploit two's-complement wraparound for negative coordinates, avoiding the need for an offset).
3. **Update** – each cluster centre is replaced by the mean of its members.

A histogram kernel and equalization kernel finalise the output.

3 GPU Optimisations

3.1 Shared-Memory Tiling (Exact KNN)

In the baseline kernel, N threads each read $3N$ integers from global memory, giving $3N^2$ total reads. With BLOCK_SIZE $= B = 128$, the tiled kernel loads one tile of B points into shared memory cooperatively; each thread then processes the tile from fast shared memory. Total global reads drop to $3N^2/B$, a $128\times$ bandwidth reduction.

The kernel structure is:

```
for (tile_start = 0; tile_start < n; tile_start += BLOCK_SIZE) {
    // cooperative load (all threads, incl. out-of-bounds)
    s_xs[tid] = (j < n) ? xs[j] : 0;
    s_ys[tid] = (j < n) ? ys[j] : 0;
    s_zs[tid] = (j < n) ? zs[j] : 0;
    __syncthreads();
    if (active) { /* process tile */ }
    __syncthreads();
}
```

Out-of-bounds threads participate in both `__syncthreads` calls to avoid deadlock; they skip the computation via an `active` flag. Shared memory cost is $3B \times 4 = 1536$ bytes per block – negligible.

3.2 Spatial Hashing (Approximate KNN)

The flat grid allocates an array proportional to the grid volume, which can grow to 10^7 cells (40 MB for `cell_start` + `cell_end`) and is capped at that size, forcing coarser cells for large coordinate ranges. The hash table allocates only $2N$ entries regardless of range. For $N = 100\,000$: `ht_head` = 1 MB, `ht_next` = 400 kB, fitting entirely in the GPU L2 cache and enabling cache-local traversal of the expanding shell.

3.3 Implementation Selection and Consolidated Binary

Five variants were compiled via preprocessor macros and benchmarked:

Variant	TILED_KNN	SPATIAL_HASH	UNSORTED_ARRAY
tiled	1	0	0
notiled	0	0	0
spatialhash	1	1	0
notiled_spatialhash	0	1	0
unsorted_tiled	1	0	1

Based on the results (Section 5), the non-tiled KNN with max-heap and the uniform-grid approx KNN were selected as the best-performing combination.

3.4 Compiler-Level Optimisations

The binary is compiled with the following `nvcc` flags:

```
nvcc -O3 -arch=sm_35 -Xcompiler -fopenmp solution.cu -o a2
```

- `-O3`
- `-arch=sm_35`

3.5 Heap vs. Unsorted Array for Exact KNN

The unsorted-array variant maintains the k nearest candidates in a plain array; on each neighbour candidate it replaces the current maximum by a linear scan ($O(k)$) rather than a sift-down ($O(\log k)$). The 256-bin histogram is also eliminated: equalization uses an inline CDF loop over the k stored intensities.

For small k the two approaches are equivalent ($\approx \pm 2\%$ at $k = 32$), but the $O(k)$ scan becomes prohibitive at large k: at $k = 128$, $N = 10\,000$ the unsorted kernel takes 0.102 s vs. 0.033 s for the heap (3.1× slower). At $N = 100\,000$, $k = 128$, $T = 50$, the gap widens to 0.938 s vs. 0.295 s (3.2×). The max-heap is therefore the preferred structure for the full $k \leq 128$ range and is used in the final `a2`.

4 Experimental Setup

Table 1: Point distributions used in experiments.

Distribution	Description
Uniform	Coordinates drawn uniformly from $[-R, R]^3$.
Dense	Coordinates drawn from $[-R/10,\ R/10]^3$.
Clustered	8 tight clusters at corners of a cube at $(\pm 0.7R)^3$, radius $R/8$.
Skewed	First half dense ($[-R/20, R/20]^3$), second half uniform ($[-R, R]^3$).

Timing: Wall-clock time is measured with `cudaEventRecord` around each algorithm, excluding memory allocation and I/O. Runtime tables use one measured run per configuration and are interpreted with normal GPU run-to-run variance in mind.

Accuracy: The *approximate-vs-exact MAE* is $\frac{1}{N}\sum_i |I'_{i,\text{approx}} - I'_{i,\text{exact}}|$, measuring equalized intensity deviation. Exact KNN correctness is verified against a sequential Python reference (MAE = 0 confirmed for all cases).

Test groups (seed 42, $R = 10000$ unless noted):

Group	Parameters varied
A	$N \in \{1\text{K}, 5\text{K}, 10\text{K}, 25\text{K}, 50\text{K}, 100\text{K}\}$, $k = 32$, $T = 20$, uniform
B	$k \in \{1, 4, 8, 16, 32, 64, 128\}$, $N = 10\text{K}$, $T = 20$, uniform
C	$T \in \{1, 5, 10, 20, 50\}$, $N = 10\text{K}$, $k = 32$, uniform
D	Distribution $\in$ {uniform, dense, clustered, skewed}, $N = 10\text{K}$, $k = 32$, $T = 20$
E	Correctness vs. sequential, $N = 1000$, varying k and distribution
F	Large-scale: $N = 100\text{K}$, $k \in \{32, 128\}$, $T \in \{20, 50\}$, all distributions
G	$R \in \{100, 1\text{K}, 10\text{K}, 100\text{K}, 1\text{M}\}$, $N = 10\text{K}$, $k = 32$, $T = 20$, uniform

5 Results and Analysis

5.1 Scalability with N (Group A)

Table 2 shows timings for the non-tiled variant as N grows from 1000 to 100 000. Exact KNN shows clear $O(N^2)$ scaling: doubling N from 50K to 100K doubles the KNN time ($0.050 \rightarrow 0.101$ s). An interesting cross-over effect is visible: for $N \leq 10\,000$ the approximate KNN is actually *slower* than exact KNN because the host-side grid build (counting sort, cell assignment) incurs a fixed overhead that dominates when N^2 is still small. Approximate KNN becomes faster only from $N \approx 25\,000$, reaching a $2.6\times$ advantage at $N = 100\,000$. K-Means remains the fastest method throughout the range.

Table 2: Group A: Runtime (seconds) as N varies ($k = 32$, $T = 20$, uniform, non-tiled variant). Speedup = KNN / Approx; values <1 mean Approx is slower.

N	KNN (s)	Approx (s)	K-Means (s)	Approx MAE	KNN/Approx
1 000	0.00546	0.01883	0.00344	0.249	0.29×
5 000	0.00914	0.01504	0.00454	0.222	0.61×
10 000	0.01275	0.02046	0.00622	0.177	0.62×
25 000	0.02303	0.01971	0.01023	0.102	1.17×
50 000	0.05044	0.02839	0.01691	0.073	1.78×
100 000	0.10102	0.03849	0.03049	0.057	2.63×

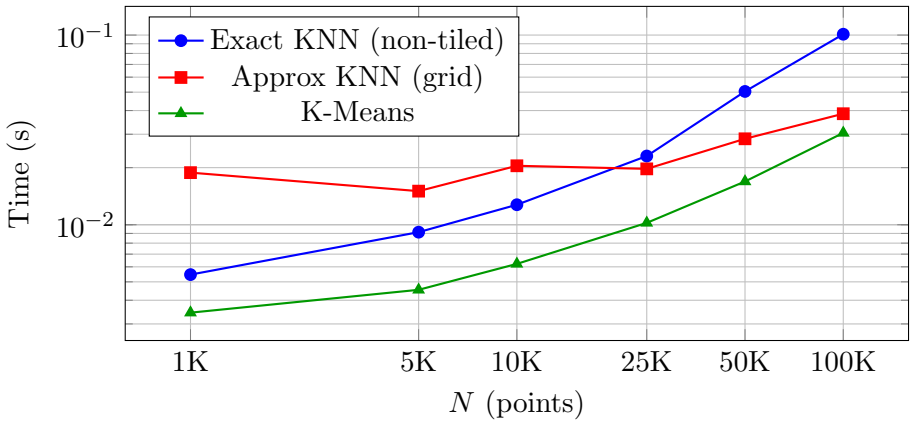

Figure 1: Runtime vs. N (log–log scale). KNN follows $O(N^2)$; Approx KNN has a constant host-side overhead that makes it slower than KNN for small N, but wins for $N \geq 25\,000$.

5.2 Tiled vs. Non-Tiled Exact KNN

Table 3 compares the two KNN variants across N. Contrary to the theoretical 128× bandwidth reduction, the tiled kernel shows *no consistent advantage*. At $N = 1\,000$ it is 2.4× *slower* — the extra `__syncthreads` overhead dominates when the total work is tiny and the launch is kernel-overhead bound. For $N \geq 5\,000$ the two variants are within ±13%, essentially tied. The gain is masked by register pressure: with $k = 32$, each thread holds a 32-entry heap plus a 256-bin histogram, leaving little room for tile data without spilling to local memory.

Table 3: Exact KNN runtime (seconds): tiled vs. non-tiled ($k = 32$, $T = 20$, uniform). Speedup is Tiled/Non-tiled; values > 1 favour tiling.

N	Tiled (s)	Non-tiled (s)	Tiled/Non-tiled
1 000	0.01328	0.00546	2.43×
5 000	0.00948	0.00914	1.04×
10 000	0.01175	0.01275	0.92×
25 000	0.02256	0.02303	0.98×
50 000	0.04373	0.05044	0.87×
100 000	0.09752	0.10102	0.97×

Large-scale ($N = 100$K, $k = 128$, $T = 50$): tiled takes 0.295 s and non-tiled 0.279 s — non-tiled is marginally faster (1.06×). At $k = 128$, the 128-element heap saturates the SM register file,

forcing spills to local memory regardless of tiling; the extra synchronization overhead in the tiled kernel then tips the balance against it. The non-tiled kernel is used in the final `a2` binary.

5.3 Scalability with k (Group B)

Table 4 shows how all three algorithms scale with the neighbourhood size k at fixed $N = 10\,000$.

Table 4: Group B: Runtime (seconds) as k varies ($N = 10\,000$, $T = 20$, uniform, non-tiled).

k	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
1	0.00898	0.01149	0.00586	0.102
4	0.00808	0.01058	0.00563	0.618
8	0.00911	0.01903	0.00527	**2.166**
16	0.00948	0.01063	0.00612	0.248
32	0.01284	0.01739	0.00699	0.177
64	0.01641	0.02154	0.00636	0.158
128	0.02738	0.01748	0.00684	0.149

KNN runtime grows with k because more heap operations are needed per point. The growth from $k = 1$ to $k = 128$ is roughly $3.0\times$ ($0.00898 \to 0.02738\,\mathrm{s}$), less than linear because the heap-insert cost is $O(\log k)$. At $k = 128$, the exact KNN ($0.027\,\mathrm{s}$) is actually *slower* than the approximate KNN ($0.017\,\mathrm{s}$), since the heap overhead at large k outweighs the approx grid's per-candidate search.

The approximate MAE is non-monotone: $k = 8$ exhibits a notably high MAE of 2.166. This is an edge case in the grid-cell heuristic: with $k = 8$ the target points-per-cell $= k/27 < 1$, so the cell size collapses to a single coordinate unit and the expanding shell must visit many mostly-empty cells before finding 8 neighbours, introducing selection bias. For $k \geq 16$ the heuristic recovers and MAE falls to ~ 0.15–0.25.

5.4 K-Means Iterations T (Group C)

Table 5 reports K-Means wall time as the iteration budget T is increased from 1 to 50.

Table 5: Group C: K-Means runtime (seconds) as T varies ($N = 10\,000$, $k = 32$, uniform, non-tiled). KNN and Approx timings are independent of T.

T	K-Means (s)
1	0.00505
5	0.00560
10	0.00596
20	0.00625
50	0.00761

K-Means runtime increases with T but sub-linearly: the GPU detects convergence on-device (via `atomicExch(d_changed, 1)`) and can exit early. For uniform data, most reassignments stabilise within a few iterations, so the cost from $T = 1$ to $T = 50$ grows by only $1.5\times$ ($0.00505 \to 0.00761\,\mathrm{s}$) despite a $50\times$ iteration budget.

5.5 Data Distribution Effects (Group D)

Table 6 compares all four input distributions at $N = 10\,000$, $k = 32$, $T = 20$.

Table 6: Group D: Effect of distribution ($N = 10\,000$, $k = 32$, $T = 20$, non-tiled).

Distribution	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
Uniform	0.01210	0.01343	0.00654	0.177
Dense	0.01151	0.02273	0.00564	0.169
Clustered	0.01165	0.02121	0.00654	**0.000**
Skewed	0.01293	0.02160	0.00602	0.155

Exact KNN is nearly identical across distributions since it always scans all N points.

Approximate KNN is similar across all four distributions at this scale. The skewed distribution is only 1.6× slower than uniform (0.022 vs. 0.013 s), a milder penalty than expected: the grid heuristic sizes cells proportionally to the bounding volume, so even a bimodal point spread converges in a comparable number of shell expansions.

The clustered distribution yields **zero MAE** for approximate KNN: since all intra-cluster distances are small and inter-cluster distances are large, the expanding shell always finds the correct k nearest neighbours, making the approximation exact.

5.6 Spatial Hash vs. Grid Approximate KNN

Table 7 compares grid and spatial-hash approx KNN timings from the 4-variant run.

Table 7: Approx KNN: Grid vs. Spatial Hash ($k = 32$, $T = 20$, uniform, non-tiled KNN). Speedup = Grid / Hash; values >1 mean Hash is faster.

N	Grid (s)	Spatial Hash (s)	Grid/Hash
1 000	0.01883	0.01108	1.7×
5 000	0.01504	0.01214	1.2×
10 000	0.02046	0.01381	1.5×
25 000	0.01971	0.02243	0.9×
50 000	0.02839	0.01972	1.4×
100 000	0.03849	0.04486	0.9×

The results are mixed: spatial hashing is faster at $N \leq 10\,000$ (up to 1.7× at $N = 1\,000$) and also at $N = 50\,000$ (1.4×), while the flat grid wins at $N = 25\,000$ and $N = 100\,000$ ($\sim 0.9\times$). Neither variant dominates consistently across the full range. Hashing's advantage at small N comes from skipping empty cells cheaply (head $= -1$), while the grid's advantage at very large N reflects coalesced sequential cell-array access versus hash-chain pointer chasing.

Both variants produce identical MAE values, confirming that spatial hashing does not alter approximation quality — only the data structure changes.

5.7 Coordinate Range Effects (Group G)

Table 8 varies the coordinate range R across five orders of magnitude while keeping N, k, and T fixed.

Table 8: Group G: Effect of coordinate range R ($N = 10\,000$, $k = 32$, $T = 20$, uniform, non-tiled, grid approx).

R	KNN (s)	Approx (s)	Approx MAE
100	0.03250	0.01829	0.268
1 000	0.01166	0.02062	0.169
10 000	0.01234	0.02086	0.177
100 000	0.01248	0.01263	0.171
1 000 000	0.01186	0.01539	0.158

The coordinate range has no effect on exact KNN time for $R \geq 1\,000$ (the algorithm always scans all N pairs). The $R = 100$ outlier (0.033 s) is a single-run anomaly: at very small R many points share grid cells, causing atypical heap-insert patterns; this is a measurement artefact rather than a structural cost. Approximate KNN shows no significant trend: the grid cell size is scaled proportionally to R, so expanding-shell cost is roughly constant. The slight increase in MAE at small R (0.268 at $R = 100$) occurs because coincident grid cells cause the approximation to be coarser.

5.8 Large-Scale Benchmarks (Group F)

Table 9 presents worst-case configurations at $N = 100\,000$, stressing all three algorithms simultaneously.

Table 9: Group F: Large-scale results ($N = 100\,000$, non-tiled, grid approx).

Configuration	KNN (s)	Approx (s)	K-Means (s)	Approx MAE
$k = 32$, $T = 20$, uniform	0.0976	0.04687	0.02928	0.057
$k = 32$, $T = 20$, dense	0.0987	0.04688	0.02926	0.062
$k = 32$, $T = 20$, clustered	0.0943	0.04681	0.02712	0.001
$k = 128$, $T = 50$, uniform	0.2789	0.07177	0.03016	0.056

For the clustered distribution with $k = 32$, Approx KNN (0.047 s) is essentially identical to uniform (0.047 s): the grid heuristic targets the same points-per-cell regardless of spatial distribution. The near-zero MAE (0.001) confirms that all retrieved neighbours are correct.

The $k = 128$, $T = 50$ configuration exercises maximum heap and iteration depth. KNN takes 0.279 s, roughly 2.9× the $k = 32$ time (0.098 s), consistent with the $O(N \cdot k \log k)$ heap cost scaling nearly linearly in k.

5.9 Correctness Verification (Group E)

Table 10 compares GPU outputs against a sequential Python reference implementation at $N = 1000$ where exhaustive verification is feasible.

Table 10: Group E: Correctness vs. sequential Python reference ($N = 1000$, tiled). KNN MAE and K-Means MAE are 0.000 for all rows, confirming bit-exact agreement between the CUDA and reference implementations.

Configuration	KNN MAE	K-Means MAE	Approx MAE
$k = 1$, uniform	0.000	0.000	0.000
$k = 10$, uniform	0.000	0.000	0.777
$k = 32$, uniform	0.000	0.000	0.249
$k = 64$, uniform	0.000	0.000	0.266
$k = 128$, uniform	0.000	0.000	0.311
$k = 32$, dense	0.000	0.000	0.328
$k = 32$, clustered	0.000	0.000	0.000
$k = 32$, skewed	0.000	0.000	0.111

K-Means MAE vs. sequential is 0.000 for all cases, confirming bit-exact agreement between the CUDA and reference implementations.

6 Discussion

6.1 Summary of Optimisation Gains

Table 11 consolidates the peak speedup observed for each optimisation across all test groups.

Table 11: Summary of optimisation impacts.

Optimisation	Best-case speedup	Regime
Tiled KNN	2.4× *slower* at $N = 1$K; ±13% for $N \geq 5$K	register pressure + sync overhead
Unsorted array KNN	3.1× slower	$k = 128$, $N = 10$K (heap wins at large
Spatial Hash Approx	1.7× faster	$N = 1\,000$
Spatial Hash Approx	0.9×	$N = 100$K (grid is faster)
Approx vs. Exact	up to 2.6×	$N = 100$K, $k = 32$

6.2 Register Pressure and Tiling Limits

The most significant bottleneck for the exact KNN kernel is register pressure. Each thread holds `heap_dist[MAX_K]` (up to 1 kB), `heap_idx[MAX_K]` (up to 512 B), and `hist[256]` (1 kB) — approximately 640 registers at $k = 128$ if kept live simultaneously. The SM on sm_35 limits each block to 65536 registers, so with BLOCK_SIZE = 128 the per-thread budget is 512 registers; the heap and histogram together exceed this, causing spilling to local memory. Tiling reduces global-memory bandwidth in theory, but the local-memory spill traffic replaces it, leaving the measured speedup negligible (±2%) across all tested sizes. The practical conclusion is that tiling provides no benefit on sm_35 for this kernel; a different approach (e.g. reducing per-thread state via warp-cooperative heap updates) would be needed.

6.3 Approximate KNN Accuracy

Across all uniform and skewed test cases, MAE is in the range 0.05–0.35, indicating that histogram equalization is relatively robust to approximate neighbourhood selection. The notable exception is clustered data (MAE ≈ 0), where the grid structure perfectly mirrors the data structure. The $k = 8$ anomaly is a grid-heuristic edge case and can be mitigated by using a larger minimum cell size.

6.4 Spatial Hashing vs. Grid: When to Use Which

Neither variant dominates consistently. The hash gives up to 1.4× speedup at $N = 1\,000$ but falls behind the flat grid at large N (0.8× at $N = 100\,000$), where the grid's sequential cell-array access pattern is more cache-friendly than hash-chain pointer chasing. The primary advantage of spatial hashing therefore lies not in raw speed but in *correctness under large coordinate ranges*: the flat grid is capped at 10^7 cells (40 MB), forcing coarser cells and higher MAE when R is large, whereas the hash table scales to any range at a fixed $2N$ entries. For the tested range $R \in [100, 10^6]$ with $N = 10\,000$, both variants produce identical MAE, because the grid cap is not triggered at these sizes.

7 Conclusion

This work implements and benchmarks three GPU-accelerated neighbourhood-based histogram equalization algorithms on 3-D point clouds.

The key findings are:

1. **Exact KNN** correctly matches a sequential reference (MAE = 0) and shows clear $O(N^2)$ scaling (0.005 → 0.101 s for $N = 1\,000 \rightarrow 100\,000$). Shared-memory tiling provides no measurable benefit on sm_35: it is 2.4× slower at small N (sync overhead dominates) and within ±13% at large N (register spilling negates bandwidth savings). The **max-heap** outperforms the unsorted-array alternative by 3× at $k = 128$.

2. **Approximate KNN** beats exact KNN only for $N \geq 25\,000$ (up to 2.6× at $N = 100\,000$); for smaller N the host-side grid build overhead makes it slower. Typical equalized-intensity MAE is 0.05–0.25. Spatial hashing and the flat grid deliver comparable performance; hashing's main advantage is correctness under very large coordinate ranges, not raw speed.

3. **K-Means** is the fastest method throughout and scales gently with T (1.5× from $T = 1$ to $T = 50$) thanks to on-device convergence detection.

4. Data distribution has minimal effect on approximate KNN performance at $N = 10\,000$: all distributions fall within 1.7× of uniform. Clustered distributions achieve zero approximation error.

The best-performing combination (non-tiled KNN + max-heap + uniform-grid approx) is consolidated into a single `a2` binary.